Week 3: Building the Backend: Express 1

Problem Statement

This week we are building the backend for **ThreadHive**, a social media platform where users can create communities (subreddits), post discussion topics (threads), add comments, vote and interact with content.

Your job is to build the **REST API layers** that powers this platform using **Node.js**, **Express.js**, and **MongoDB (Mongoose)**.

Project Setup and Structure

1. Download and extract the assignment starter code from the zip file provided on Olympus.
2. Initialize a new Git repository (either by running `git init` in the terminal or by using the VS Code Git extension).
3. As you make changes throughout this tutorial, remember to regularly commit your work.

The main folders in the starter code are as follows:

```
src/
├── models/        → Mongoose schemas (User, Subreddit, Thread)
├── services/      → Core Business logic (DB queries, mutations)
├── controllers/   → Request/response handling
├── routes/        → Express route definitions
└── scripts/       → Utility scripts for database seeding
```

We have already built the database models in the previous week. The data models, database connection (`db.js`), and app configuration (`src/app.js`) are already in place. The main file (`main.js`) is the entry point of the application. You need to implement the business logic and HTTP endpoints on top of them. We will do this in two parts:

	Part	Focus	AI Usage
Part 1	Manual Implementation	Subreddit API (Service → Controller → Route)	No AI Tools
Part 2	AI-Assisted Development	Thread API Implementation + Custom Instructions + Prompt Files	GitHub Copilot

In **Part 1** you will build foundational understanding by writing every line yourself. In **Part 2** we will use GitHub Copilot workflows to see how AI accelerates the same process.

By the end of this session, you will be able to:

- Build complete Express.js REST API endpoints from scratch
- Configure GitHub Copilot **Custom Instructions** to enforce project-wide coding standards
- Use Copilot's **Plan Mode** and **Agent Mode** to design and implement APIs
- Create reusable **Prompt Files** for common developer tasks like code review

Part 1: Manual Implementation — Subreddit API

In Part 1 you will implement the following three endpoints for the Subreddit API:

Method	Endpoint	Description
GET	`/api/subreddits`	Fetch all subreddits
POST	`/api/subreddits`	Create a new subreddit
GET	`/api/subreddits/:id`	Fetch a subreddit and its threads

You will build them from the bottom up: **Service → Controller → Route**. You might choose to build the entire pipeline for one endpoint first, and then move to the next.

Step 1.1: Implement the Subreddit Service

Open `src/services/subredditService.js`. This file handles all direct database interactions using Mongoose. Implement the three functions described in the comments.

Tips:

- All functions should be `async` — Mongoose methods return Promises
- `Subreddit.findById(id)` returns `null` automatically if the ID doesn't exist
- `.populate("author")` replaces the ObjectId reference with the actual User document

---

Step 1.2: Implement the Subreddit Controller

Open `src/controllers/subredditController.js`. Controllers receive HTTP requests, call service functions, and send back HTTP responses with the correct status codes.

Use this standard response envelope for **all** responses:

```
// Success
{ "success": true, "message": "...", "data": { ... } }

// Error
{ "success": false, "message": "..." }
```

HTTP status codes to use:

Situation	Status Code
Successfully fetched data	`200`
Resource created	`201`
Missing required fields	`400`
Resource not found	`404`
Duplicate / conflict	`409`
Unexpected server error	`500`

Step 1.3: Wire Up the Routes

Open `src/routes/subreddits.js`. Register the three routes using Express Router and connect them to your controller functions.

> **Note:** Paths here are relative. The `/api/subreddits` prefix is already applied in `src/app.js`, which is pre-configured for you.

The full request flow for context:

```
HTTP Request
    → Express app (app.js)                                ← already set up
    → Router (routes/subreddits.js)                       ← your work
    → Controller (controllers/subredditController.js)     ← your work
    → Service (services/subredditService.js)              ← your work
    → MongoDB (via Mongoose model)                        ← already set up
```

Step 1.4: Test Your Implementation

Let us test each endpoint using Postman. We will seed the database and then start the server. Before doing this, set up your MongoDB connection and port number in a `.env` file at the root of your project:

```
MONGODB_URI=your_mongodb_connection_string
PORT=3000
```

1. To seed the database with initial data, first install the dependencies with `npm install`, then run:

   ```
   npm run populate
   ```

This runs `src/scripts/populate_db.js` and inserts sample Users, Subreddits, and Threads into MongoDB. Copy an ObjectId of a User — you'll need it when creating a subreddit.

2. Start the Express server:

```
npm run dev
```

3. Test the implemented endpoints using **Postman**:

Method	URL	Body (JSON)
GET	`http://localhost:3000/api/subreddits`	—
POST	`http://localhost:3000/api/subreddits`	`{ "name": "react", "description": "Discussions about React", "author": "<user_id>" }`
GET	`http://localhost:3000/api/subreddits/<subreddit_id>`	—

> **Note:** Replace `<user_id>` with the ObjectId of a User from your database, and `<subreddit_id>` with the ObjectId of a Subreddit from your database.

Part 2: AI-Assisted Implementation — Thread API with GitHub Copilot

In Part 2, you will learn how to:

> - use **Plan Mode** for high-level API design decisions
> - configure **Custom Instructions** so Copilot understands your project's standards
> - use **Plan and Agent Mode** for phased, structured backend code generation and testing with Postman
> - create reusable **Prompt Files** for repeatable developer tasks

You will implement all the Thread API endpoints using the same architecture from Part 1 — but this time, Copilot does the heavy lifting. The Thread API has **five endpoints** to implement:

Method	Endpoint	Description
GET	`/api/threads`	Fetch all threads
GET	`/api/threads/:id`	Fetch a single thread
POST	`/api/threads`	Create a new thread
PUT	`/api/threads/:id`	Update a thread

Method	Endpoint	Description
DELETE	`/api/threads/:id`	Delete a thread

Step 2.1: Design API endpoints (Plan Mode in a NEW PROJECT)

IMPORTANT NOTE: Create a **NEW project** to work on this use case. This is to avoid the API implementation from Part 1 influencing the Agent's design decisions. Since this is a design exercise, it is NOT necessary to setup the project with a full Express backend — we just need the models as context for the API design.

We will revert back to the ThreadHive codebase in the next steps when we move to implementation.

Let us use Copilot's **Plan Mode** to design the API architecture.

1. Copy the models folder from the ThreadHive application in the new project. We will use these models as context for our API design since they represent the data structure we need to work with.
2. Open the Chat Panel and start a **new chat in Plan Mode**
3. Use a strong reasoning model (Claude Opus 4.5+ or GPT-5.2+). Make sure to select the 'High' thinking effort.
4. Use this prompt:

```
You are an expert Backend architect. Help me design complete REST API
endpoints for a Reddit-style social platform.

CONTEXT:
- The models/ directory contains MongoDB collections for Users,
Subreddits and Threads in the application.

YOUR TASK:
1. Analyze the existing Mongoose models to understand data
relationships
2. Design a complete API architecture that includes:
   - All necessary endpoints grouped by resource
   - HTTP methods and paths
   - Request/response formats
   - Query parameters for filtering, sorting, pagination
   - Relationship handling (nested resources vs separate endpoints)

3. Consider RESTful best practices in your design
4. Present the design in a structured format with:
   - Endpoint inventory (table format)
   - Detailed endpoint specifications
```

```
Show me two different architectural approaches and compare them. Then
recommend the best approach with justification.
```

5. Notice how the Agent suggests different architectural approaches to implement the backend. Carefully study the different approaches and the recommended design. You can iterate on the design by asking follow-up questions in the same conversation.

Step 2.2: Custom Instructions (AGENTS.md)

IMPORTANT NOTE: Now switch back to your original ThreadHive project. All the remaining steps will be done in the ThreadHive codebase.

Custom Instructions allow you to define project-specific guidelines that Copilot follows across your entire codebase. Think of them as a persistent set of rules for your project — so you don't have to repeat the same instructions in every chat.

Typical instructions include coding standards, architectural patterns, API response formats, error handling conventions, and anything else you want Copilot to consistently adhere to.

GitHub Copilot will automatically look for a `copilot-instructions.md` file in the `.github` folder of your repository. If it finds one, it will use those instructions as context for all code generation.

However most coding agents (including Copilot) also recognize a distinct instructions file called `AGENTS.md` located in the root of the repository. We will use a `AGENTS.md` for our custom instructions; this will allow our instructions to be picked up by a wide variety of agents, not just GitHub Copilot.

Exercise: Create Your Custom Instructions File

1. Move the file `AGENTS-template.md` from the `resources` folder into the project root and rename it to `AGENTS.md`
2. Review these custom instructions and note the key sections.

Key Takeaway: Custom instructions eliminate the need to repeat project context in every prompt and ensure consistency across all AI-generated code. Sometimes however, you might find that the generated code doesn't fully adhere to your instructions (it is NOT guaranteed that the generated code will always follow each and every instruction). In such cases, you can iterate with the agent to refine the output or even manually edit the code if the deviations are minor.

Commit your `AGENTS.md` file to the repository. Treat it as a critical part of your project documentation that defines how AI should interact with your codebase.

Step 2.3: Implement the Thread API (Plan & Agent Mode)

We have already saved the list of endpoints for our backend in `resources/finalized-apis.md`. Such a list can be easily built by asking the agent to generate it based on the models and previous API design decisions.

Let us now use a phased approach to implement these endpoints using GitHub Copilot: first generate a plan in Plan Mode, then execute it phase by phase in Agent Mode.

Generate Implementation Plan (Plan Mode)

1. Open Chat Panel → **new chat in Plan Mode**
2. Use this prompt:

```
Create a 3-phase implementation plan for Thread endpoints in
#file:resources/finalized-apis.md

Split by http-methods:
- Phase 1: GET endpoints
- Phase 2: POST and PUT endpoints
- Phase 3: DELETE endpoint

For each phase, list: endpoints to implement, required files, and key
considerations.
```

3. **Review the generated plan carefully** before moving to implementation — adjust if needed

Phase 1: GET Endpoints (Agent Mode)

1. Switch to **Agent Mode** and enter the following prompt:

```
Implement Phase 1 of the plan
```

2. Review the generated code carefully.

Phase 2: POST and PUT Endpoints (Agent Mode)

1. Once Phase 1 is implemented, enter the following prompt in the same conversation:

```
Implement Phase 2 of the plan
```

2. Review the generated code carefully.

Phase 3: DELETE Endpoint (Agent Mode)

1. Once Phase 2 is implemented, enter the following prompt in the same conversation:

```
Implement Phase 3 of the plan
```

2. Review the generated code carefully.

Create Postman Collections to test all Thread API endpoints.

We can now test that all the Thread API endpoints are working correctly using Postman. We can use Agents to quickly generate a Postman collection with all the necessary requests pre-filled.

1. In the same Agent conversation, enter the following prompt:

```
Generate Postman collections for all the Thread API endpoints
implemented here. Make sure you use the Collection format v2.1.0. Save
the collection as a JSON file in the resources folder. Generate sample
requests for each endpoint, but do not generate the response.
```

2. The Agent will generate a Postman collection JSON file in the `resources` folder. Import this collection into Postman and test each endpoint to ensure they are working correctly. You will need to update the request bodies with valid data from your database (e.g., valid user IDs, subreddit IDs, thread IDs) before sending the requests.

3. After testing that all the endpoints are working correctly, commit your changes to the repository.

Step 2.4: Reusable Prompt Files

As you develop with Copilot, you'll find yourself typing similar prompts repeatedly for common tasks. **Prompt Files** let you save these as reusable templates invoked with a slash command directly in the Chat Panel. Instead of having to repeatedly type similar instructions for a task like code review, you can create a prompt file once and reuse it whenever needed.

Create a Code Review Prompt

1. Create a folder `.github/prompts` in your project root

2. Move the `review-code.prompt.md` file in the `resources` folder into `.github/prompts`

3. Carefully review the prompt file and understand how it is structured.

4. Test it by running the reviewer on a file. In the Chat Panel type:

```
/review-code #file:src/controllers/threadController.js
```

5. The /review-code command will attach the contents of the `review-code.prompt.md` file along with the user's input, and send it to the model. The model will then generate a code review based on the prompt instructions and the input code.